

Rapport technique — ScholaritéApp

Projet de fin de module : **Spring** Licence Génie Logiciel — Faculté des Sciences de Kénitra (UIT)
— 2025-2026

1. Introduction

Ce projet consiste à réimplémenter, avec l'écosystème **Java / Spring**, un cas d'usage de **gestion de scolarité** initialement proposé sous Django. L'application permet de gérer les **étudiants, enseignants, filières, modules, inscriptions** et **notes**, avec une authentification et une gestion fine des droits par rôle.

Le projet est composé de deux parties :

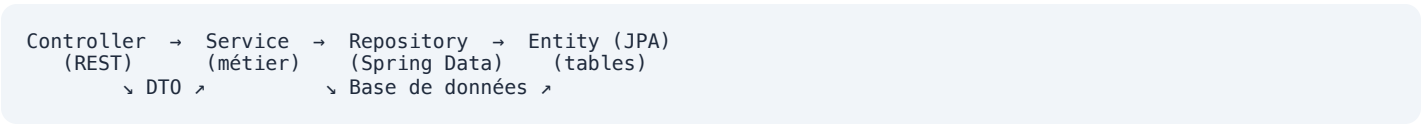
- un **backend Spring Boot** exposant une **API REST** sécurisée par **JWT** ;
- un **frontend React** (TypeScript) qui consomme cette API.

2. Architecture générale

L'application suit une architecture **client / serveur découplée** : le frontend et le backend sont deux applications indépendantes communiquant uniquement via HTTP/JSON.



Le backend est lui-même organisé en **couches** (architecture en oignon classique) :



Couche	Rôle	Exemple
Controller	Expose les endpoints REST, valide les entrées	EtudiantController
Service	Logique métier, mapping Entity ↔ DTO	EtudiantService

Couche	Rôle	Exemple
Repository	Accès aux données via Spring Data JPA	EtudiantRepository
Entity	Représentation des tables	Etudiant
DTO	Objets de transfert (jamais exposer les entités)	EtudiantDto, EtudiantRequest

3. Choix techniques

3.1 Backend

Technologie	Rôle	Justification
Spring Boot 3.3	Framework applicatif	Standard de l'industrie, auto-configuration, démarrage rapide
Spring Web (MVC)	Exposition de l'API REST	Annotations @RestController, @RequestMapping
Spring Data JPA	Persistence	Réduit le code d'accès aux données (repositories)
Spring Security + JWT	Authentification / autorisation	Sécurité stateless adaptée à une API consommée par un SPA
Hibernate	ORM	Implémentation JPA par défaut
H2 / MySQL	Base de données	H2 pour le dev (zéro installation), MySQL pour la prod
springdoc-openapi	Documentation (Swagger)	Documentation interactive auto-générée
Lombok	Réduction du boilerplate	Génère getters/setters/builders
JUnit 5 + MockMvc	Tests	Tests d'intégration des endpoints

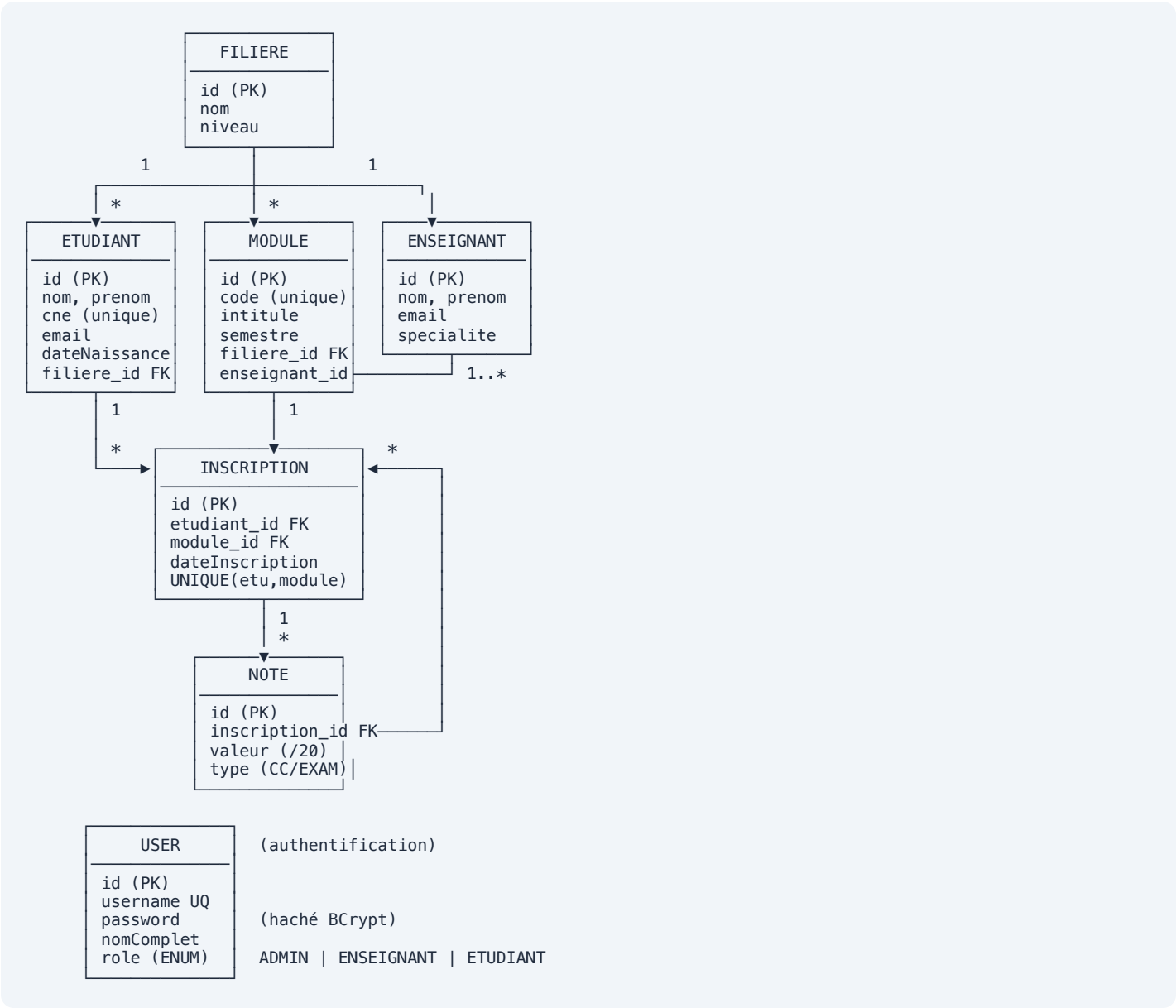
3.2 Frontend

Technologie	Rôle	Justification
React 19 + TypeScript	Bibliothèque UI	Composants réutilisables, typage statique
Vite	Build / serveur de dev	Démarrage instantané, proxy intégré

Technologie	Rôle	Justification
React Router	Navigation SPA	Routes protégées par rôle
Axios	Client HTTP	Intercepteurs (injection automatique du JWT)
Tailwind CSS	Style	Interface moderne, rapide à développer

4. Modèle de données

4.1 Schéma relationnel



4.2 Relations principales

- Une **Filière** contient plusieurs **Étudiants** et plusieurs **Modules** (1..*).

- Un **Enseignant** est responsable de plusieurs **Modules** (1..*).
 - L'**Inscription** matérialise l'association *plusieurs-à-plusieurs* entre **Étudiant** et **Module** (contrainte d'unicité sur le couple).
 - Une **Note** est rattachée à une **Inscription** (donc à un couple étudiant/module).
 - Le **User** porte l'authentification et le **rôle**.
-

5. Sécurité

L'authentification est **stateless** et repose sur des **JSON Web Tokens (JWT)**.

1. L'utilisateur s'authentifie via `POST /api/auth/login`.
2. Le serveur vérifie les identifiants (mot de passe haché avec **BCrypt**) et renvoie un **token JWT** signé (HMAC) contenant le nom d'utilisateur et le rôle.
3. À chaque requête, le frontend envoie le token dans l'en-tête `Authorization: Bearer <token>`.
4. Un filtre `JwtAuthenticationFilter` (exécuté une fois par requête) valide le token et place l'utilisateur dans le `SecurityContext`.
5. Les autorisations par rôle sont appliquées via `@PreAuthorize` sur les méthodes des contrôleurs (ex. seul **ADMIN** peut créer un étudiant ; **ADMIN** et **ENSEIGNANT** peuvent saisir des notes).

Gestion des erreurs : un `GlobalExceptionHandler (@RestControllerAdvice)` transforme les exceptions en réponses JSON cohérentes (404, 400 + erreurs de validation, 401).

6. API REST

L'API respecte les conventions REST (noms de ressources au pluriel, verbes HTTP, codes de statut). Elle est entièrement documentée par **Swagger UI** (`/swagger-ui.html`).

Ressource	Endpoints
Auth	<code>POST /api/auth/login</code> , <code>POST /api/auth/register</code>
Étudiants	<code>GET/POST/PUT/DELETE /api/etudiants</code> , <code>GET /api/etudiants/{id}/notes</code>
Enseignants	<code>GET/POST/PUT/DELETE /api/enseignants</code>
Filières	<code>GET/POST/PUT/DELETE /api/filieres</code>
Modules	<code>GET/POST/PUT/DELETE /api/modules</code>

Ressource	Endpoints
Inscriptions	GET/POST/DELETE /api/inscriptions
Notes	GET/POST/PUT/DELETE /api/notes
Dashboard	GET /api/dashboard/stats

7. Frontend

Le frontend est une **Single Page Application** :

- **Authentification** : page de login, stockage du JWT dans `localStorage`, contexte React (`AuthContext`) partageant l'utilisateur courant.
- **Routes protégées** : un composant `ProtectedRoute` redirige vers `/login` si l'utilisateur n'est pas connecté et masque/filtre les actions selon le rôle.
- **Client API** : instance Axios avec intercepteurs (ajout automatique du token, déconnexion automatique sur réponse 401).
- **Pages** : tableau de bord (statistiques), et pages CRUD pour chaque ressource (tableaux + formulaires en modale).

Pendant le développement, Vite **proxifie** les appels `/api` vers le backend (port 8080), ce qui évite tout problème de CORS.

8. Déploiement et hébergement

Composant	Hébergement recommandé
Frontend (build statique)	Sous-domaine o2switch, Netlify ou Vercel
Backend (jar Java)	Render, Railway ou Fly.io
Base de données	MySQL (o2switch) ou PostgreSQL géré

⚠ Un hébergement mutualisé PHP (type o2switch) ne peut pas exécuter une application Java : il convient pour le frontend statique, mais le backend doit être déployé sur une plateforme supportant la JVM.

9. Conclusion

Ce projet a permis de mettre en œuvre les principaux modules de l'écosystème Spring (**IoC / injection de dépendances, Spring MVC, Spring Data JPA, Spring Security**) au sein d'une application réelle, et de les articuler avec un client moderne **React / TypeScript** consommant une API REST. Il illustre la comparaison entre l'approche Django (monolithique, full-stack Python) et l'approche Spring (backend Java découplé + SPA JavaScript).

Comptes de démonstration : admin/admin123, prof/prof123, etudiant/etud123.